The three planner studies, re-tested exhaustively and proven in Lean

2026-10-26 · `research/ic_planner_exhaustive_20261026` · protocol frozen in `0ca110399` before any session. 14 ecbench sessions (3,328 records, all verified, 14 of 14 audits OK, with 12 exact replays each) and 16 curve-free null cases. 2,428 band verdicts were checked by Lean's `decide`.

Question

The owner asked for three earlier results to be re-tested more exhaustively and proven in Lean:

- **the residual collector** on prime curves;
- **the §23 Koblitz descent** against the `ic plan` pipeline;
- **the binary/Koblitz weight rule**, together with the prime weight rule.

The re-test used four new prime curves (15.7–36.3 bits), four held-out random binary curves and six Koblitz curves (36.6–47.2 bits). Every prediction and every decision rule was fixed before the first session.

Result

family	prediction	outcome
residual	R1–R5	all held, including the risky R3 (res4 cheaper than the plan at w = 1 at p37)
null	N1 (at most 2 of 16 intervals exclude 1)	held: 1 of 16
Koblitz routing	K1 (descent dearer than pipeline at all six sizes)	held: 1.31–2.04×
	K2 (both ratios to rho grow with size)	failed narrowly: pipeline/rho dips from k59 to k61
prime rule	W1	failed on its w = 0 clause; the w = 1 part held on 4 of 4
binary rule	W2	failed: $d_0 + 1$ cheaper at w = 1 on hb19
Koblitz rule	W3	failed: $0.71 \cdot c_0$ dearer at every weight

The residual collector is cheaper than the plan across the whole band from 36 bits

curve	$\log_2 r$	S plan	S res2	res2/plan w=0	w=0.3	w=1	res4/plan w=1
p16	15.70	10.77	6.48	0.602 [0.543, 0.666]	1.566 [1.360, 1.803]	2.768 [2.365, 3.239]	4.919 [4.317, 5.604]
p18	17.70	10.23	4.84	0.473 [0.438, 0.511]	1.406 [1.269, 1.558]	2.448 [2.188, 2.739]	4.713 [4.215, 5.270]
p36	36.00	46.37	2.94	0.063 [0.056, 0.071]	0.277 [0.245, 0.314]	**0.493 [0.433, 0.561]**	**0.888 [0.817, 0.965]**
p37	36.25	49.17	2.84	0.058 [0.053, 0.063]	0.251 [0.229, 0.276]	**0.445 [0.405, 0.489]**	**0.865 [0.802, 0.933]**

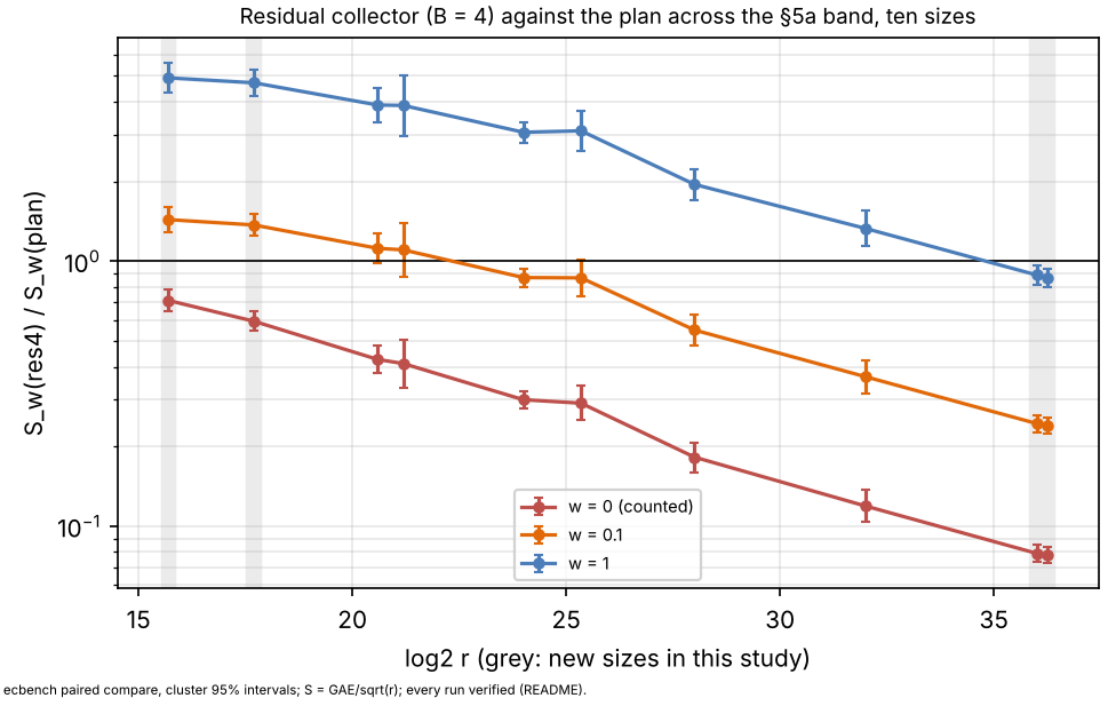


Figure 1: res4/plan over ten sizes (six earlier, four new). The w = 1 ratio falls from 4.9 to 0.87 and crosses 1 between 32 and 36 bits.

The gain is generic.

- **Null:** on `Z/r` with a random key, the collector costs the same (N1, 15 of 16 intervals contain 1).
- **Exponent:** its fitted ten-size exponent is 0.460, close to rho's 1/2, against 0.619 for the plan.

Distance remaining. At 36 bits, res2 is 2.6–2.7× `rho.negation` in counted S and 23–24× at w = 1. For the plan itself the figures are 41–47× and 47–55×. So the collector is ranked below the plan across the band only as a generic collision search; it is not an index-calculus improvement.

All three weight rules fail out of sample and are withdrawn (`ic-planner/4`)

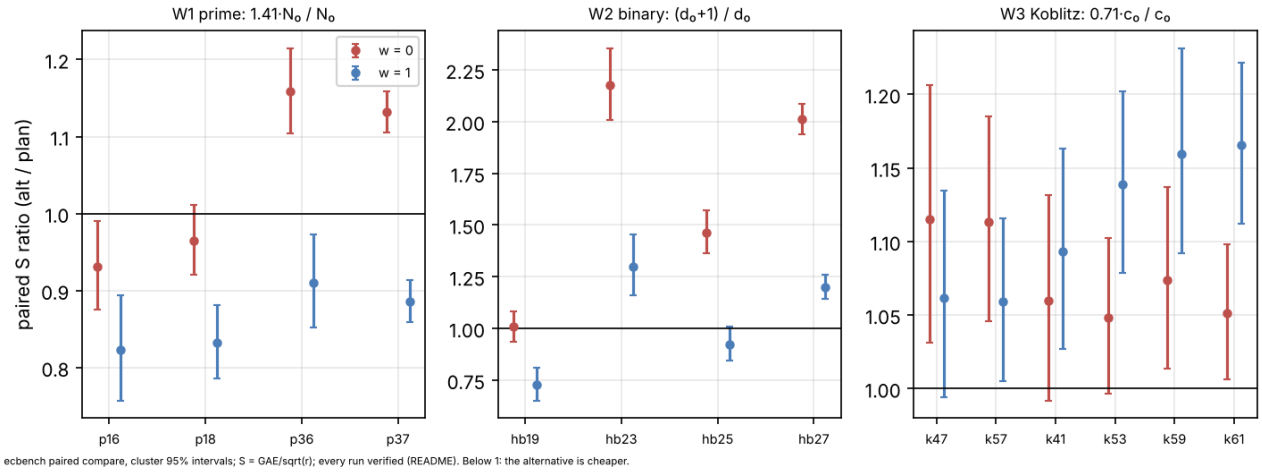


Figure 2: Held-out paired ratios, alternative over plan, at w = 0 (red) and w = 1 (blue).

- **Prime (W1).** At w = 1, 1.41·N_0 costs 0.82–0.91× N_0 on 4 of 4 curves, each interval below 1. But it is also cheaper at w = 0 at 15.7 and 17.7 bits, so the preregistered control (the w = 0 ratio at least 1 on 3 of 4 curves) fails.
- **Binary (W2).** At w = 1, d_0 + 1 costs 0.726× d_0 on hb19 and 0.922× on hb25.
- **Koblitz (W3).** At 36.6–47.2 bits, 0.71·c_0 costs 1.06–1.17× c_0 at w = 1. The earlier hold-out had found 0.80–0.88× at 18–26 bits.
- **Decision.** Under the protocol's decision rule `ic plan` keeps N_0, d_0 and c_0 at every weight and warns about the withdrawn rule. A size-dependent rule would need its own protocol.

Koblitz routing and the curve-free null

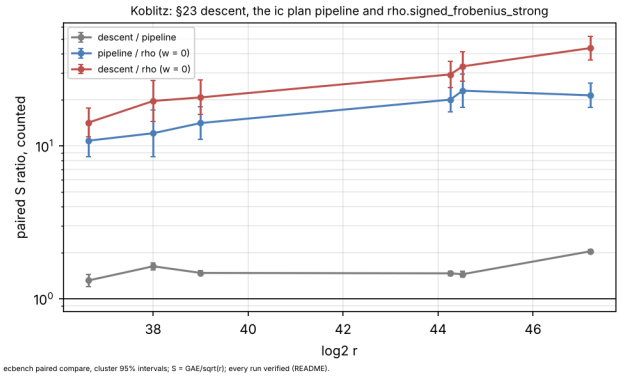


Figure 3: Descent/pipeline 1.31–2.04×; both far above rho.

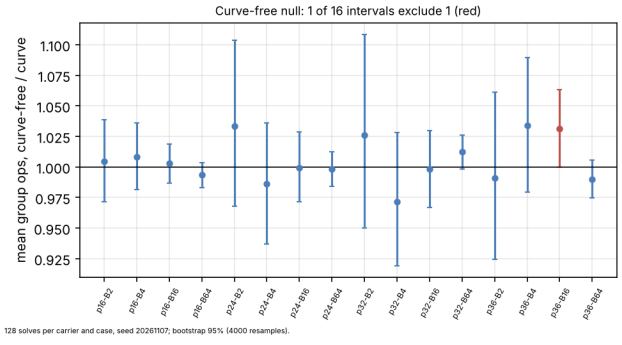


Figure 4: Null: 1 of 16 intervals excludes 1.

What Lean proves (`lean/`, Lean core v4.34.1)

- **`Band.lean`.** Priced cost is affine in the weight. A reversal between w = 0 and w = 1 forces one exact crossing weight in (0, 1), with fixed orders on either side. As the weight rises, the optimum moves only to arms with no more native work.
- **`KeyedInvariance.lean`.** An algorithm that sees elements only through `add`, `neg`, equality and a key has an identical transcript (observations, cost and answer) on carriers joined by an injective, key-preserving homomorphism. Corollary: the residual collector on E equals the collector on `Z/r` with E's key pulled back.
- **`Certificates.lean`.** 2,428 verdicts from 39 band files of this study and the four earlier ones: 2,010 orders, 58 exact crossing weights and 360 argmin facts. Each restates exact integer band sums and is closed by `decide`. Every reported `m*(w)` of the earlier derivations is re-derived and matches.

- **Audit.** No `sorry`, `admit` or `native_decide`. The certificates use no axioms; the main theorems use only propext, Classical.choice and Quot.sound. `leanchecker --fresh` passes. CI runs `planner-proofs-lean.yml`.
- **Not in Lean.** The bootstrap intervals; the correspondence between the Rust code and the model, which is checked by reading the code and by ecbench's own sum test.

Deviations

- **No resume checkpoint.** There is no `CHECKPOINT_UPLOAD_TOKEN`; the owner said to proceed.
- **Card 3, two failed attempts.**
  - The first crashed when the session's Lean install filled the disk. Its partial outputs were deleted unread.
  - The second was killed by the harness at its 30-minute limit. Three complete sessions were kept, and the partial R-p37 was deleted unread and rerun on card 4.
- **Card 8.** The main session ran it unchanged from its own shell: the executor's 30-minute window could not hold K-k61.
- **Commits between cards.** They were data only; every session ran the same three binaries, bound by sha256.
- **Null intervals.** They use a Python bootstrap with 4,000 resamples.
- **Local Lean build.** It used `lean -o` without lake; CI uses lake.

No leaderboard row moves: the validation gate needs an independent validator.